PL/SQL Variables

A variable is a meaningful name which facilitates a programmer to store data temporarily during the execution of code. It helps you to manipulate data in PL/SQL programs. It is nothing except a name given to a storage area. Each variable in the PL/SQL has a specific data type which defines the size and layout of the variable's memory.

A variable should not exceed 30 characters. Its letter optionally followed by more letters, dollar signs, numerals, underscore etc.

How to declare variable in PL/SQL

You must declare the PL/SQL variable in the declaration section or in a package as a global variable. After the declaration, PL/SQL allocates memory for the variable's value and the storage location is identified by the variable name.

Syntax for declaring variable:

Following is the syntax for declaring variable:

1. variable_name [CONSTANT] datatype [NOT NULL] [:= | **DEFAULT** initial_value]
   EXAMPLES:-

   int Roll_no[not null]; OR ROLL_NO INT [NOT NULL]

   int grand_total=500;

Here, variable name is a valid identifier in PL/SQL and datatype must be valid PL/SQL data type. A data type with size, scale or precision limit is called a constrained declaration. The constrained declaration needs less memory than unconstrained declaration.

Example:

Radius Number := 5;

Date_of_birth date;

Declaration Restrictions:

In PL/SQL while declaring the variable some restrictions hold.

- Forward references are not allowed i.e. you must declare a constant or variable before referencing it in another statement even if it is a declarative statement.
  val number := Total - 200;
  Total number := 1000;
  The first declaration is illegal because the TOTAL variable must be declared before using it in an assignment expression.
- Variables belonging to the same datatype cannot be declared in the same statement.
  N1, N2, N3 Number;

  It is an illegal declaration.

Naming rules for PL/SQL variables

The variable in PL/SQL must follow some naming rules like other programming languages.

- The variable_name should not exceed 30 characters.
- Variable name should not be the same as the table table's column of that block.
- The name of the variable must begin with ASCII letter. The PL/SQL is not case sensitive so it could be either lowercase or uppercase. For example: v_data and V_DATA refer to the same variables.
- You should make your variable easy to read and understand, after the first character, it may be any number, underscore (_) or dollar sign ($).
- NOT NULL is an optional specification on the variable.

Initializing Variables in PL/SQL

Every time you declare a variable, PL/SQL defines a default value NULL to it. If you want to initialize a variable with other value than NULL value, you can do so during the declaration, by using any one of the following methods.

- The DEFAULT keyword
- The assignment operator

```
1. counter binary_integer := 0;
2. greetings varchar2(20) DEFAULT 'Hello ';
```

You can also specify NOT NULL constraint to avoid NULL value. If you specify the NOT NULL constraint, you must assign an initial value for that variable.

You must have a good programming skill to initialize variable properly otherwise, sometimes program would produce unexpected result.

Example of initializing variable

Let's take a simple example to explain it well:

```
1. DECLARE
2.    a integer := 30;
3.    b integer := 40;
4.    c integer;
5.    f real;
6. BEGIN
7.    c := a + b;
8.    dbms_output.put_line ('Value of c: ' || c);
9.    f := 100.0/3.0;
10.   dbms_output.put_line('Value of f: ' || f);
11. END;
```

After the execution, this will produce the following result:

```
Value of c: 70
Value of f: 33.33333333333333333

PL/SQL procedure successfully completed.
```

Variable Scope in PL/SQL:

PL/SQL allows nesting of blocks. A program block can contain another inner block. If you declare a variable within an inner block, it is not accessible to an outer block. There are two types of variable scope:

- **Local Variable**: Local variables are the inner block variables which are not accessible to outer blocks.
- **Global Variable:** Global variables are declared in outermost block.

Example of Local and Global variables

Let's take an example to show the usage of Local and Global variables in its simple form:

```
1.  DECLARE
2.   -- Global variables
3.     num1 number := 95;
4.     num2 number := 85;
5.  BEGIN
6.     dbms_output.put_line('Outer Variable num1: ' || num1);
7.     dbms_output.put_line('Outer Variable num2: ' || num2);
8.     DECLARE
9.        -- Local variables
10.     num1 number := 195;
11.      num2 number := 185;
12.    BEGIN
13.       dbms_output.put_line('Inner Variable num1: ' || num1);
14.       dbms_output.put_line('Inner Variable num2: ' || num2);
15.    END;
16. END;
17. /
```

After the execution, this will produce the following result:

```
Outer Variable num1: 95
Outer Variable num2: 85
Inner Variable num1: 195
Inner Variable num2: 185

PL/SQL procedure successfully completed.
```

Variable Attributes:

When you declare a PL/SQL variable to hold the column values, it must be of correct data types and precision, otherwise error will occur on execution. Rather than hard coding the data type and precision of a variable. PL/SQL provides the facility to declare a variable without having to specify a particular data type using %TYPE and %ROWTYPE attributes. These two attributes allow us to specify a variable and have

that variable data type be defined by a table/view column or a PL/SQL package variable.

A % sign servers as the attribute indicator. This method of declaring variables has an advantage as the user is not concerned with writing and maintaining code.

Following are the types of Variable Attributes in PL/SQL.

- **%TYPE:**

The %TYPE attribute is used to declare variables according to the already declared variable or database column. It is used when you are declaring an individual variable, not a record. The data type and precision of the variable declared using %TYPE attribute is the same as that of the column that is referred from a given table. This is particularly useful when declaring variables that will hold database values. When using the %TYPE keyword, the name of the columns and the table to which the variable will correspond must be known to the user. These are then prefixed with the variable name. If some previously declared variable is referred then prefix that variable name to the %TYPE attribute.

The syntax for declaring a variable with %TYPE is:

1. <var_name> <tab_name>.<column_name>%TYPE;

Where <column_name> is the column defined in the <tab_name>.

Consider a declaration.

SALARY EMP.SAL % TYPE;

This declaration will declare a variable SALARY that has the same data type as column SAL of the EMP table.

Example:

```
1. DECLARE
2. SALARY EMP.SAL % TYPE;
3. ECODE EMP.empno % TYPE;
4. BEGIN
5. Ecode :=&Ecode;
```

6. **Select** SAL **into** SALARY **from** EMP **where** EMPNO = ECODE;
7. dbms_output.put_line('Salary of ' || ECODE || 'is = || salary');
8. **END**;

After the execution, this will produce the following result:

```
Enter value for ecode: 7499
Salary of 7499 is = 1600
PL/SQL procedure successfully completed.
```

- **%ROWTYPE:**

The %ROWTYPE attribute is used to declare a record type that represents a row in a table. The record can store an entire row or some specific data selected from the table. A column in a row and corresponding fields in a record have the same name and data types.

The syntax for declaring a variable with %ROWTYPE is:

1. <var_name> <tab_name>.ROW%TYPE;

Where <variable_name> is the variable defined in the <tab_name>.

Consider a declaration.

EMPLOYEE EMP. % ROW TYPE;

This declaration will declare a record named EMPLOYEE having fields with the same name and data types as that of columns in the EMP table. You can access the elements of EMPLOYEE record as

EMPLOYEE.SAL := 10000;

EMPLOYEE.ENAME := 'KIRAN';

Example:

```
1. DECLARE
2. EMPLOYEE EMP. % ROW TYPE;
3. BEGIN
4. EMPLOYEE.EMPNO := 2092;
    EMPLOYEE.ENAME := 'Sanju';
5. Insert into EMP where (EMPNO, ENAME) Values (employee.empno, employee.enam
   e);
6. dbms_output.put_line('Row Inserted');
7. END;
```

After the execution, this will produce the following result:

```
Row Inserted
PL/SQL procedure successfully completed.
```

Advantages:

- If you don't know the data type at the time of declaration. The data type assigned to the associated variables will be determined dynamically at run time.
- If the data type of the variable you are referencing changes the %TYPE or %ROWTYPE variable changes at run time without having to rewrite variable declarations. For example: if the ENAME column of an EMP table is changed from a VARCHAR2(10) to VRACHAR2(15) then you don't need to modify the PL/SQL code.

PL/SQL Constants

A constant is a value used in a PL/SQL block that remains unchanged throughout the program. It is a user-defined literal value. It can be declared and used instead of actual values.

Let's take an example to explain it well:

Suppose, you have to write a program which will increase the salary of the employees upto 30%, you can declare a constant and use it throughout the program. Next time if you want to increase the salary again you can change the value of constant than the actual value throughout the program.

Syntax to declare a const

1. constant_name CONSTANT datatype := VALUE;

- **Constant_name:**it is the name of constant just like variable name. The constant word is a reserved word and its value does not change.
- **VALUE:** it is a value which is assigned to a constant when it is declared. It can not be assigned later.

Example of PL/SQL constant

Let's take an example to explain it well:

```
DECLARE
   -- constant declaration
   pi constant number := 3.141592654;
   -- other declarations
   radius number(5,2);
   dia number(5,2);
   circumference number(7, 2);
   area number (10, 2);
BEGIN
   -- processing
   radius := 9.5;
   dia := radius * 2;
   circumference := 2.0 * pi * radius;
   area := pi * radius * radius;
   -- output
   dbms_output.put_line('Radius: ' || radius);
   dbms_output.put_line('Diameter: ' || dia);
   dbms_output.put_line('Circumference: ' || circumference);
   dbms_output.put_line('Area: ' || area);
END;
/
```

After the execution of the above code at SQL prompt, it will produce the following result:.

```
Radius: 9.5
Diameter: 19
Circumference: 59.69
```

4. Area: 283.53
5. Pl/SQL **procedure** successfully completed.

PL/SQL Literals

Literals are the explicit numeric, character, string or boolean values which are not represented by an identifier. For example: TRUE, NULL, etc. are all literals of type boolean. PL/SQL literals are case-sensitive. There are following kinds of literals in PL/SQL:

- Numeric Literals
- Character Literals
- String Literals
- BOOLEAN Literals
- Date and Time Literals

Example of these different types of Literals:

Literals	Examples
Numeric	75125, 3568, 33.3333333 etc.
Character	'A' '%' '9' ' ' 'z' '('
String	Hello JavaTpoint!
Boolean	TRUE, FALSE, NULL, NOT NULL etc.
Date and Time	'26-11-2002' , '2012-10-29 12:01:01'